

PDF 2: Código de Programación

A continuación se presentan snippets de código en múltiples lenguajes para probar la retención de indentación y símbolos.

Snippet en Python

```
# -*- coding: utf-8 -*-
"""
Python: lo raro acá no es un símbolo, es la AUSENCIA de símbolos.
No hay { } que delimiten bloques – la indentación ES la sintaxis.
Además, Python permite identificadores con caracteres unicode (¡tildes incluidas!).
"""

from dataclasses import dataclass

@dataclass
class Jugador:
    nombre: str
    país: str          # ← identificador normal, pero string con tilde
    dorsal: int

def clasificación(jugadores: list[Jugador]) -> dict[str, int]:
    tabla = {}
    for j in jugadores:          # el for no lleva llaves, ni fin, ni end
        if j.país not in tabla:   # el bloque se define por la indentación
            tabla[j.país] = 0
            tabla[j.país] += j.dorsal
    return tabla

# Un poco de azúcar sintáctica: list comprehension + walrus operator (:=)
def resumen(jugadores: list[Jugador]) -> str:
    if (n := len(jugadores)) == 0:
        return "sin jugadores ☐"
    nombres = [j.nombre for j in jugadores if j.dorsal < 10]
    return f"{n} jugadores, {len(nombres)} con dorsal < 10 → {nombres}"

if __name__ == "__main__":
    equipo = [
        Jugador("Ñoño Fernández", "Argentina", 9),
        Jugador("François Dupont", "France", 7),
        Jugador("Björn Müller", "Deutschland", 4),
    ]
    print(clasificación(equipo))
    print(resumen(equipo))
```

Snippet en Smalltalk

```
"Smalltalk: lo raro acá es que TODO, absolutamente todo, es un objeto
y toda acción es enviar un mensaje a un objeto. No hay operadores
'especiales',
hasta el '+' es un mensaje binario. Los comentarios van entre comillas
dobles.
Los símbolos (#símbolo) y los caracteres especiales ($ñ, $€) son ciudadanos
de
primera clase."

Object subclass: #Jugador
  instanceVariableNames: 'nombre país dorsal'
  classVariableNames: ''
  package: 'Mundial'.

Jugador class >> nombre: unNombre país: unPaís dorsal: unDorsal [
  | j |
  j := self new.
  j setNombre: unNombre país: unPaís dorsal: unDorsal.
  ^ j
].

Jugador >> setNombre: unNombre país: unPaís dorsal: unDorsal [
  nombre := unNombre.
  país := unPaís.
  dorsal := unDorsal.
].

Jugador >> saludar [
  "Mensaje unario: se envía sin argumentos, con un simple punto y aparte"
  Transcript showCr: nombre, ' (#', dorsal printString, ') - ', país, ' -
;vamos €quipo!'.
].

"Envío de mensajes en cascada (;), característica muy propia de Smalltalk"
| equipo |
equipo := OrderedCollection new.
equipo
  add: (Jugador nombre: 'Ñandú Pérez' país: 'Argentina' dorsal: 10);
  add: (Jugador nombre: 'Åsa Löfgren' país: 'Sverige' dorsal: 8);
  yourself.

equipo do: [ :cadaJugador | cadaJugador saludar ].
```

Snippet en Prolog

```
% Prolog: lo raro acá es que no "programás" pasos, declarás HECHOS y REGLAS,
% y es el motor de inferencia (unificación + backtracking) el que "resuelve"
% las preguntas por vos. No hay asignación de variables tradicional.
% Los átomos con tildes/ñ hay que encerrarlos entre comillas simples.

% --- Hechos ---
jugador('Ñoño Fernández', 'argentina', 9).
jugador('François Dupont', 'francia', 7).
jugador('Björn Müller', 'alemania', 4).
jugador('Åsa Löfgren', 'suecia', 8).
```

```

campeón('argentina', 2022).
campeón('francia', 2018).
campeón('alemania', 2014).

% --- Regla: un jugador es "campeón vigente" si su país ganó el último
mundial ---
% El operador ':-' se lee "si". La coma es "y". Todo termina en punto.
campeón_vigente(Jugador) :-
    jugador(Jugador, País, _),
    campeón(País, Año),
    Año >= 2022.

% --- Regla recursiva clásica: ¿cuántos jugadores hay en la lista? ---
cantidad([], 0).
cantidad(_|Resto, N) :-
    cantidad(Resto, N0),
    N is N0 + 1.

% Consultas de ejemplo (se tipearían en el intérprete):
% ?- campeón_vigente(X).
%    X = 'Ñño Fernández'.
% ?- jugador(Nombre, 'francia', Dorsal).
%    Nombre = 'François Dupont', Dorsal = 7.

```

Snippet en Haskell

```

-- Haskell: lo raro acá es que es puro (sin efectos secundarios ocultos),
-- perezoso (una lista puede ser infinita y no pasa nada) y fuertemente
tipado
-- con inferencia de tipos. Con {-# LANGUAGE UnicodeSyntax #-} hasta se puede
-- escribir λ en vez de "\" y → en vez de "->".

{-# LANGUAGE UnicodeSyntax #-}

module Mundial where

data Jugador = Jugador
    { nombre :: String
    , país   :: String
    , dorsal :: Int
    } deriving (Show, Eq)

-- Pattern matching en la definición de la función, sin ifs explícitos
esTitular :: Jugador → Bool
esTitular j
    | dorsal j <= 11 = True
    | otherwise      = False

-- Lambda con sintaxis unicode (λ y →) en vez de \ y ->
dorsales :: [Jugador] → [Int]
dorsales = map (λj → dorsal j)

-- Lista infinita de fibonacci, gracias a la evaluación perezosa –
-- esto en un lenguaje "estricto" jamás terminaría de construirse
fibonacci :: [Integer]
fibonacci = 0 : 1 : zipWith (+) fibonacci (tail fibonacci)

```

```

main :: IO ()
main = do
  let equipo =
    [ Jugador "Ñoño Fernández" "Argentina" 9
    , Jugador "François Dupont" "France" 7
    , Jugador "Åsa Löfgren" "Sverige" 8
    ]
  putStrLn $ "Titulares → " ++ show (filter esTitular equipo)
  putStrLn $ "Dorsales → " ++ show (dorsales equipo)
  putStrLn $ "Fibonacci (10) → " ++ show (take 10 fibonacci)

```

Snippet en Mercado

```

<!DOCTYPE html>
<!--
  HTML: lo raro acá es que NO es un lenguaje de programación (no hay lógica,
  no hay variables) sino de MARCADO: describe estructura, no comportamiento.
  Además usa "entidades" (&ntilde; &acute; &euro;) para representar
  caracteres
  especiales cuando la codificación no está garantizada.
-->
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Selecci&ouml;n &ntilde;and fuerte</title>
</head>
<body>
  <header>
    <h1>Plantel Mundialista &euro;2026</h1>
  </header>

  <main>
    <section id="jugadores">
      <ul>
        <li data-dorsal="9">&Ntilde;o&ntilde;o Fern&aacute;andez – Argentina</li>
        <li data-dorsal="7">Fran&ccedil;ois Dupont – France</li>
        <li data-dorsal="8">&Aring;sa L&ouml;fgren – Sverige</li>
      </ul>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 – Hecho con cari&ntilde;o y demasiadas etiquetas
    </p>
  </footer>
</body>
</html>

```

Snippet en Estilos

```

/*
  CSS: tampoco es un lenguaje de programación clásico – no hay funciones
  propias (aunque calc() y las custom properties se le acercan) y su lógica
  central es LA CASCADA y LA ESPECIFICIDAD: quién le "gana" a quién.
  El pseudo-elemento ::before puede inyectar contenido, incluso con

```

```

    caracteres especiales, directo en el CSS con content: "...".
  */

:root {
  --color-argentina: #75AADB;
  --color-francia: #0055A4;
  --color-alemania: #FFCE00;
}

.jugador {
  font-family: "Segoe UI", sans-serif;
  padding: 0.5rem 1rem;
  border-left: 4px solid var(--color-argentina);
}

.jugador[data-país="Sverige"] {
  border-left-color: #006AA7;
}

.jugador::before {
  content: "★ ";          /* carácter especial directo en el CSS */
}

.jugador::after {
  content: " – ¡vamos!";  /* signos de apertura/cierre españoles, sin drama */
}

@media (max-width: 600px) {
  .jugador {
    font-size: 0.9rem;
  }
}

```

Snippet en Componente

```

// JS/JSX: lo raro acá es que JSX mezcla HTML *dentro* del propio código JS
// (no es válido JS puro, necesita un compilador tipo Babel) y que JS tiene
// azúcar sintáctica rarísima: destructuring, template literals con `${}`,
// spread operator (...), y optional chaining (?).

import { useState } from "react";

const jugadores = [
  { nombre: "Ñoño Fernández", país: "Argentina", dorsal: 9 },
  { nombre: "François Dupont", país: "France", dorsal: 7 },
  { nombre: "Åsa Löfgren", país: "Sverige", dorsal: 8 },
];

// Destructuring + template literal con interpolación `${...}`
const presentar = ({ nombre, país, dorsal }) =>
  `#${dorsal} → ${nombre} (${país})`;

export default function Plantel() {
  const [seleccionado, setSeleccionado] = useState(null);

  return (
    <section className="plantel">

```

```

<h2>Selección Ñandú Fuerte</h2>
<ul>
  {jugadores.map((j) => (
    <li
      key={j.dorsal}
      onClick={() => setSeleccionado(j)}
    >
      {presentar(j)}
    </li>
  )})}
</ul>

  {seleccionado?.nombre && (
    <p>Elegiste a: {seleccionado.nombre} – ¡qué crack! ☐☐</p>
  )}
</section>
  );
}

```

Snippet en Jugador

```

// Java: lo raro acá es lo VERBOSO que es todo – no existe una sola línea
// suelta de código, TODO tiene que vivir dentro de una clase, hasta el
// método main(). Además es fuertemente tipado y compilado (no interpretado
// como Python), así que muchos errores se detectan antes de correr nada.

import java.util.List;
import java.util.stream.Collectors;

public class Jugador {
  private final String nombre;
  private final String país;      // los identificadores unicode están
  permitidos
  private final int dorsal;

  public Jugador(String nombre, String país, int dorsal) {
    this.nombre = nombre;
    this.país = país;
    this.dorsal = dorsal;
  }

  public String presentar() {
    return String.format("#%d → %s (%s) – ¡crack total!", dorsal, nombre,
    país);
  }

  public static void main(String[] args) {
    List<Jugador> equipo = List.of(
      new Jugador("Ñoño Fernández", "Argentina", 9),
      new Jugador("François Dupont", "France", 7),
      new Jugador("Åsa Löfgren", "Sverige", 8)
    );

    // Streams + lambdas: la parte "moderna" y menos verbosa de Java
    String resumen = equipo.stream()
      .filter(j -> j.dorsal < 10)
      .map(Jugador::presentar)
      .collect(Collectors.joining("\n"));
  }
}

```

```

        System.out.println(resumen);
    }
}

```

Snippet en Mundial

```

// Go: lo raro acá es la concurrencia integrada en el propio lenguaje:
// goroutines (go func()) y channels para comunicarlás, sin necesidad de
// librerías externas. Además, las funciones pueden devolver MÚLTIPLES
// valores a la vez (muy poco común en otros lenguajes de esta lista),
// y el manejo de errores es explícito (if err != nil), no con try/catch.

package main

import "fmt"

type Jugador struct {
    Nombre string
    País   string // identificador unicode permitido también en Go
    Dorsal int
}

// Retorno múltiple: valor + error, patrón idiomático de Go
func buscarPorDorsal(equipo []Jugador, dorsal int) (Jugador, error) {
    for _, j := range equipo {
        if j.Dorsal == dorsal {
            return j, nil
        }
    }
    return Jugador{}, fmt.Errorf("no se encontró jugador con dorsal %d",
dorsal)
}

func saludar(j Jugador, canal chan string) {
    canal <- fmt.Sprintf("¡Vamos %s! #%d - %s", j.País, j.Dorsal,
j.Nombre)
}

func main() {
    equipo := []Jugador{
        {"Ñoño Fernández", "Argentina", 9},
        {"François Dupont", "France", 7},
        {"Åsa Löfgren", "Sverige", 8},
    }

    canal := make(chan string, len(equipo))

    // Se lanzan goroutines concurrentes, cada una manda su saludo al
channel
    for _, j := range equipo {
        go saludar(j, canal)
    }

    for range equipo {
        fmt.Println(<-canal)
    }
}

```

```

        if jugador, err := buscarPorDorsal(equipo, 7); err == nil {
            fmt.Println("Encontrado:", jugador.Nombre)
        }
    }
}

```

Snippet en Mundial

```

// C#: lo raro acá, comparado con Java, es lo integrado que está LINQ
// (consultas estilo SQL directamente sobre colecciones) y las "propiedades"
// (get/set) que evitan escribir getters/setters manuales a lo Java.
// También permite interpolación de strings con $"..." y unicode sin drama.

using System;
using System.Collections.Generic;
using System.Linq;

namespace Mundial
{
    public class Jugador
    {
        public string Nombre { get; set; }
        public string País { get; set; } // propiedad auto-implementada
        public int Dorsal { get; set; }

        public string Presentar() => $"#{Dorsal} → {Nombre} ({País}) – ¡a
ganar!";
    }

    public class Program
    {
        public static void Main(string[] args)
        {
            var equipo = new List<Jugador>
            {
                new() { Nombre = "Ñoño Fernández", País = "Argentina", Dorsal
= 9 },
                new() { Nombre = "François Dupont", País = "France", Dorsal =
7 },
                new() { Nombre = "Åsa Löfgren", País = "Sverige", Dorsal =
8 },
            };

            // LINQ: consulta declarativa, muy parecida a SQL
            var titulares = from j in equipo
                            where j.Dorsal <= 11
                            orderby j.Dorsal
                            select j.Presentar();

            foreach (var línea in titulares)
                Console.WriteLine(línea);
        }
    }
}

```


Snippet en Mundial

```
// C++: lo raro acá, comparado con casi todos los anteriores, es que EXPONE
// la memoria: punteros, referencias, gestión manual (o semi-manual con
// smart pointers). Además permite templates (genéricos "de verdad", con
// generación de código en tiempo de compilación) y sobrecarga de operadores,
// algo que muy pocos lenguajes de esta lista permiten tan libremente.

#include <iostream>
#include <vector>
#include <string>
#include <memory>

struct Jugador {
    std::string nombre;
    std::string país;    // sí, los identificadores/strings unicode entran sin
drama
    int dorsal;

    // Sobrecarga del operador << para poder hacer std::cout << jugador
    friend std::ostream& operator<<(std::ostream& os, const Jugador& j) {
        os << "#" << j.dorsal << " → " << j.nombre << " (" << j.país << ") –
¡vamos!";
        return os;
    }
};

// Template genérico: funciona para cualquier tipo comparable, no solo
Jugador
template <typename T>
T máximo(const std::vector<T>& valores, bool (*comparar)(const T&, const T&))
{
    T mejor = valores.front();
    for (const auto& v : valores) {
        if (comparar(v, mejor)) mejor = v;
    }
    return mejor;
}

int main() {
    std::vector<Jugador> equipo = {
        {"Ñoño Fernández", "Argentina", 9},
        {"François Dupont", "France", 7},
        {"Åsa Löfgren", "Sverige", 8},
    };

    // std::unique_ptr: gestión de memoria "moderna", sin new/delete manual
expuesto
    auto capitán = std::make_unique<Jugador>(equipo.front());

    for (const auto& jugador : equipo) {
        std::cout << jugador << '\n';
    }

    std::cout << "Capitán elegido → " << *capitán << '\n';
    return 0;
}
```

Snippet en Sql

```
# SQL – el idioma que le habla a los datos

*Lo raro acá, comparado con los 11 anteriores: SQL es **declarativo** –vos decís
QUÉ querés, no CÓMO conseguirlo (no hay loops explícitos, el motor decide el plan de ejecución). Además mezcla varios "sub-lenguajes" en uno solo: DDL (estructura), DML (datos) y DCL/TCL (permisos/transacciones). Acá van DDL, DML completo (INSERT/UPDATE/DELETE), un SELECT con JOIN + subconsulta + agregación, y un TRIGGER.*

---

## 1. DDL – Creación de la estructura

```sql
-- DDL: Data Definition Language. Define la "forma" de los datos,
-- no los datos en sí. CREATE, ALTER, DROP viven acá.

CREATE TABLE país (
 id_país SERIAL PRIMARY KEY,
 nombre VARCHAR(60) NOT NULL, -- admite tildes/ñ sin drama si la
DB usa UTF-8
 confederación VARCHAR(20) NOT NULL,
 campeón BOOLEAN DEFAULT FALSE
);

CREATE TABLE jugador (
 id_jugador SERIAL PRIMARY KEY,
 nombre VARCHAR(80) NOT NULL,
 dorsal SMALLINT CHECK (dorsal BETWEEN 1 AND 99),
 id_país INTEGER NOT NULL REFERENCES país(id_país) ON DELETE CASCADE,
 goles INTEGER DEFAULT 0,
 actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Índice extra sobre la columna de búsqueda más frecuente
CREATE INDEX idx_jugador_país ON jugador(id_país);
```

---

## 2. INSERT – Alta de datos

```sql
-- INSERT: puede cargar una fila o varias en un solo statement (multi-row insert)

INSERT INTO país (nombre, confederación, campeón) VALUES
 ('Argentina', 'CONMEBOL', TRUE),
 ('França', 'UEFA', TRUE), -- ojo: nombre con la grafía original,
con cedilla
 ('Deutschland', 'UEFA', TRUE),
 ('Sverige', 'UEFA', FALSE);

INSERT INTO jugador (nombre, dorsal, id_país, goles) VALUES
 ('Ñoño Fernández', 9, (SELECT id_país FROM país WHERE nombre =
'Argentina'), 12),
```

```

 ('François Dupont', 7, (SELECT id_país FROM país WHERE nombre =
'França'), 8),
 ('Björn Müller', 4, (SELECT id_país FROM país WHERE nombre =
'Deutschland'), 3),
 ('Åsa Löfgren', 8, (SELECT id_país FROM país WHERE nombre =
'Sverige'), 5);
\`\`\`

```

---

### ## 3. UPDATE – Modificación de datos existentes

```

\`\`\`sql
-- UPDATE siempre con WHERE, si no querés actualizar la tabla entera
-- (clásico accidente de un DBA a las 3 de la mañana).

```

```

UPDATE jugador
SET goles = goles + 1,
 actualizado_en = CURRENT_TIMESTAMP
WHERE nombre = 'Ñoño Fernández';

```

```

-- UPDATE masivo pero controlado, con subconsulta correlacionada

```

```

UPDATE país
SET campeón = TRUE
WHERE id_país IN (
 SELECT id_país FROM jugador
 GROUP BY id_país
 HAVING SUM(goles) > 10
);
\`\`\`

```

---

### ## 4. DELETE – Baja de datos

```

\`\`\`sql
-- DELETE también necesita WHERE por la misma razón que UPDATE.
-- Como jugador tiene ON DELETE CASCADE hacia país, borrar un país
-- se lleva puestos a sus jugadores automáticamente.

```

```

DELETE FROM jugador
WHERE goles = 0 AND dorsal > 20;

```

```

-- Ejemplo de baja en cascada (comentado a propósito, para no volar todo
Suecia):

```

```

-- DELETE FROM país WHERE nombre = 'Sverige';
\`\`\`

```

---

### ## 5. SELECT completo – JOIN + subconsulta + agregación

```

\`\`\`sql
-- Trae, por país, el goleador con más goles, pero solo de países campeones,
-- y solo si el equipo entero superó los 10 goles en total (HAVING).

```

```

SELECT
 p.nombre AS país,
 j.nombre AS goleador,
 j.goles,

```

```

 (SELECT COUNT(*)
 FROM jugador j2
 WHERE j2.id_país = p.id_país) AS cantidad_jugadores
FROM país p
JOIN jugador j ON j.id_país = p.id_país
WHERE p.campeón = TRUE
 AND j.goles = (
 SELECT MAX(j3.goles)
 FROM jugador j3
 WHERE j3.id_país = p.id_país
)
GROUP BY p.nombre, j.nombre, j.goles, p.id_país
HAVING SUM(j.goles) OVER (PARTITION BY p.id_país) > 10
ORDER BY j.goles DESC;
```

```

6. TRIGGER – Efecto automático ante un evento

```

```sql
-- TRIGGER: código que la propia base de datos ejecuta sola,
-- sin que ninguna aplicación se lo pida explícitamente.
-- Acá, cada vez que se actualiza el dorsal de un jugador,
-- se registra el cambio en una tabla de auditoría.

CREATE TABLE auditoría_dorsal (
 id_auditoría SERIAL PRIMARY KEY,
 id_jugador INTEGER NOT NULL,
 dorsal_viejo SMALLINT,
 dorsal_nuevo SMALLINT,
 modificado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE OR REPLACE FUNCTION registrar_cambio_dorsal()
RETURNS TRIGGER AS $$
BEGIN
 IF OLD.dorsal IS DISTINCT FROM NEW.dorsal THEN
 INSERT INTO auditoría_dorsal (id_jugador, dorsal_viejo, dorsal_nuevo)
 VALUES (OLD.id_jugador, OLD.dorsal, NEW.dorsal);
 END IF;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditar_dorsal
AFTER UPDATE OF dorsal ON jugador
FOR EACH ROW
EXECUTE FUNCTION registrar_cambio_dorsal();

-- Con esto activo, algo tan simple como esto ya queda auditado solo:
-- UPDATE jugador SET dorsal = 11 WHERE nombre = 'Åsa Löfgren';
```

```

Nota sobre caracteres especiales

Se usaron tildes, ñ y ç directamente en nombres e identificadores (`país`, `auditoría\_dorsal`, `goleón`... perdón, `goleador`), algo que la mayoría de

los motores modernos (PostgreSQL, MySQL 8 con `utf8mb4`) toleran sin drama siempre que la codificación de la base esté bien configurada – cosa que en la vida real rara vez es gratis.